



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Ingeniería en Inteligencia Artificial



CÓMPUTO PARALELO

Modelos de programación y métricas de desempeño

Entrega individual — Capítulo 3 de 7

Profesor:

José Alfredo Jiménez Benítez

Presentan:

Cruz Trejo Said Carbot

López Castro Magaly

Romero Rosales Lino Ehecatl

Ugalde Téllez Aarón

Grupo: 6BV1

Ciudad de México, a 30 de septiembre de 2026

Índice general

Índice de figuras	ii
Índice de tablas	iii
3. Modelos de programación y métricas de desempeño	1
3.1. Marco teórico	1
3.1.1. Semáforos y sincronización en lenguaje C	1
3.2. Programación con procesos en lenguaje C	2
3.2.1. Función propuesta por Aarón Ugalde Téllez	2
3.3. Análisis de la implementación con procesos	10
3.3.1. Implementación de Aarón Ugalde Téllez	10
3.4. Conclusiones del capítulo	11
3.4.1. Conclusión — Aarón Ugalde Téllez	11
3.5. Anexos	12
3.5.1. Fase 3: código de <code>fourier_aaron.c</code>	12
3.5.2. Fase 3: código de <code>verificacion_serie.py</code>	17
Referencias	19

Índice de figuras

3.1. Compilación y ejecución del programa con 100 procesos hijos en la terminal. 7

3.2. Gráfica de $F_{100}(x)$ calculada en C con 100 procesos hijos, en los 64 nodos de $x = -3.1416$
a $x = 3.1584$ 7

Índice de tablas

3.1. Función original y aproximaciones con 10 armónicos (Excel) y 100 armónicos (C, 100 procesos), en diez de los 64 nodos. 9

3.2. Error cuadrático medio respecto de $f(x)$ según la región de nodos considerada. 9

3.3. Características de la implementación con procesos. 10

3.4. Especificaciones del equipo de cómputo empleado. 11

Capítulo 3. Modelos de programación y métricas de desempeño

En este capítulo se lleva a un programa en lenguaje C el cálculo de los términos de la Serie de Fourier que se resolvió analíticamente en el capítulo 1 y se graficó en hojas de cálculo en el capítulo 2. A diferencia de la hoja de cálculo, que evalúa todo de forma secuencial, el programa reparte el trabajo entre cien procesos hijos creados con `fork()`: cada hijo calcula un armónico, los resultados se acumulan en una región de memoria compartida y un semáforo garantiza que esa acumulación se haga sin condiciones de carrera.

3.1 Marco teórico

3.1.1 Semáforos y sincronización en lenguaje C

Cuando varios procesos cooperan sobre un recurso común en memoria compartida aparece el problema de la *condición de carrera* (*race condition*): dos o más procesos leen y modifican al mismo tiempo el mismo dato, de modo que el resultado final depende del orden en que el planificador intercale sus instrucciones, y ese orden no es determinista [1]. Una instrucción tan simple como `f_x[i] += t` parece una sola operación, pero el procesador la ejecuta como tres: leer el valor, sumarle `t` y escribir el resultado. Si dos procesos leen el mismo valor antes de que cualquiera de ellos escriba, una de las dos sumas se pierde.

El mecanismo clásico para lograr la *exclusión mutua* en estas *secciones críticas* es el semáforo, propuesto por Edsger W. Dijkstra a mediados de los años sesenta [1]. Un semáforo es una variable entera administrada por el sistema operativo que sólo puede modificarse mediante dos operaciones atómicas, es decir, indivisibles:

1. **Operación P** (del holandés *Proberen*, “probar”; también *wait*): si el valor del semáforo es mayor que cero, lo decrementa y el proceso continúa; si vale cero, el proceso se bloquea y queda en la cola del semáforo hasta que otro proceso ejecute V.
2. **Operación V** (del holandés *Verhogen*, “incrementar”; también *signal* o *post*): incrementa el valor del semáforo y, si hay procesos bloqueados en su cola, despierta a uno de ellos.

Un semáforo inicializado en 1 se denomina *binario* o *mutex*: el primer proceso que ejecuta P entra a la sección crítica y cualquier otro que llegue mientras tanto se bloquea hasta que el primero ejecute V [2].

En GNU/Linux, y en general en los sistemas POSIX, los semáforos de la familia System V se administran como *conjuntos* de semáforos que el núcleo identifica mediante el mecanismo IPC (*Inter-Process Communication*) [3]. Las tres llamadas al sistema fundamentales son:

- `semget(key_t key, int nsems, int semflg)`: crea un conjunto de `nsems` semáforos, o accede a uno existente, y devuelve su identificador. Cuando los procesos que lo comparten son parientes, como ocurre aquí, basta la clave `IPC_PRIVATE`: el padre crea el conjunto antes de `fork()` y los hijos heredan el identificador.

- `semctl(int semid, int semnum, int cmd, ...)`: ejecuta operaciones de control sobre el conjunto, como fijar el valor inicial de un semáforo (`SETVAL`) o eliminar el conjunto del sistema (`IPC_RMID`).
- `semop(int semid, struct sembuf *sops, size_t nsops)`: ejecuta de manera atómica una lista de operaciones *P* o *V* descritas con la estructura `struct sembuf` del código 3.1.

```

1 struct sembuf {
2     unsigned short sem_num; // índice del semáforo dentro del conjunto (0 si
        hay uno solo)
3     short         sem_op; // -1: operación P (wait) +1: operación V (signal)
4     short         sem_flg; // banderas de control (por ejemplo, SEM_UNDO)
5 };

```

Código 3.1. Estructura `sembuf`, que describe una operación atómica sobre un semáforo System V.

Como el núcleo conserva los conjuntos de semáforos aunque el programa que los creó termine, el programa debe eliminarlos de forma explícita con `semctl(..., IPC_RMID)`; de lo contrario quedan como recursos huérfanos que se pueden listar con el comando `ipcs`.

3.2 Programación con procesos en lenguaje C

3.2.1 Función propuesta por Aarón Ugalde Téllez

Serie de Fourier por evaluar

La función asignada es la misma que se resolvió a mano en el capítulo 1 y se tabuló en el capítulo 2:

$$f(x) = -\frac{5}{6}x^2 + \frac{1}{3}x - \frac{2}{7}, \quad x \in [-\pi, \pi]. \quad (3.1)$$

Sus coeficientes de Fourier son

$$\frac{a_0}{2} = -\frac{5\pi^2}{18} - \frac{2}{7} \approx -3.0273, \quad (3.2)$$

$$a_n = \frac{10(-1)^{n+1}}{3n^2}, \quad (3.3)$$

$$b_n = \frac{2(-1)^{n+1}}{3n}, \quad (3.4)$$

y la aproximación que calcula el programa conserva los primeros cien armónicos:

$$F_{100}(x) = \frac{a_0}{2} + \sum_{n=1}^{100} \left[\frac{10(-1)^{n+1}}{3n^2} \cos(nx) + \frac{2(-1)^{n+1}}{3n} \sin(nx) \right]. \quad (3.5)$$

El capítulo anterior se limitó a diez armónicos, $F_{10}(x)$; aquí el número de términos se multiplica por diez y cada término lo calcula un proceso distinto. Los valores de x son los mismos 64 nodos de la hoja de cálculo, $x_k = -3.1416 + 0.1k$ con $k = 0, 1, \dots, 63$, es decir, de -3.1416 a 3.1584 . Conviene notar que el último nodo queda 0.0168 por encima de π , fuera del intervalo $[-\pi, \pi]$; esto tiene consecuencias visibles en la gráfica (sección 3.2.1).

Diseño de la solución con procesos

Como cada armónico n es independiente de los demás, el cálculo se reparte de forma natural: el hijo número p calcula el armónico $n = p$ en los 64 nodos. Lo único que los procesos comparten es el vector donde se suman las contribuciones, y por eso es ahí donde intervienen la memoria compartida y el semáforo. El programa sigue estos pasos:

1. El padre crea, con `shmget` y `shmat`, una región de memoria compartida para 64 valores de tipo `double` (512 bytes) y escribe en cada posición el término constante $a_0/2$.
2. Crea con `semget` un semáforo System V y lo inicializa en 1 con `semctl`, de modo que la sección crítica comienza libre.
3. Mediante un ciclo con `fork()` crea los 100 procesos hijos.
4. Cada hijo calcula su armónico en un arreglo *local* de 64 términos, sin sincronización alguna. Después ejecuta P , suma su arreglo al vector compartido, ejecuta V , se desacopla de la memoria compartida y termina.
5. El padre espera a los cien hijos con `wait()`, escribe el vector en un archivo CSV y libera la memoria compartida y el semáforo.

La sección crítica se mantuvo lo más corta posible a propósito: el cálculo trigonométrico, que es la parte costosa, se hace fuera de ella sobre datos locales de cada proceso, y el semáforo sólo protege las 64 sumas sobre el vector compartido.

Fragmentos de código explicados

El código completo se encuentra en los anexos (sección 3.5); en los fragmentos siguientes se omiten las comprobaciones de error para concentrarse en la lógica.

El código 3.2 muestra la preparación de los recursos compartidos. `shmget` con la clave `IPC_PRIVATE` crea un segmento nuevo de 512 bytes y `shmat` lo proyecta en el espacio de direcciones del padre. Como los hijos se crean después con `fork()`, heredan esa proyección y ven la misma memoria física: a diferencia del resto de la memoria del padre, que se duplica, ésta no se copia. El semáforo se crea con un solo elemento y `semctl` con `SETVAL` lo deja en 1, valor que lo convierte en un *mutex*.

```
1 size_t shm_size = NUM_PUNTOS * sizeof(double);
2 int shm_id = shmget(IPC_PRIVATE, shm_size, IPC_CREAT | 0666);
3 double *f_x = (double *)shmat(shm_id, NULL, 0);
```

```

4
5 double a0 = (-5.0 / 18.0) * (M_PI * M_PI) - (2.0 / 7.0); // a0/2
6 for (int i = 0; i < NUM_PUNTOS; i++) {
7     f_x[i] = a0;
8 }
9
10 int sem_id = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
11 union semun arg;
12 arg.val = 1; // sección crítica libre
13 semctl(sem_id, 0, SETVAL, arg);

```

Código 3.2. Creación de la memoria compartida y del semáforo en el proceso padre.

Las operaciones P y V del código 3.3 son envoltorios de `semop`. Cada uno arma una `struct sembuf` (código 3.1) sobre el semáforo 0: con `sem_op = -1` el proceso se bloquea si el semáforo vale cero, y con `sem_op = 1` libera la sección crítica.

```

1 void sem_wait_op(int sem_id) {
2     struct sembuf sb = {0, -1, 0}; // semáforo 0, operación P
3     semop(sem_id, &sb, 1);
4 }
5
6 void sem_post_op(int sem_id) {
7     struct sembuf sb = {0, 1, 0}; // semáforo 0, operación V
8     semop(sem_id, &sb, 1);
9 }

```

Código 3.3. Operaciones P (`sem_wait_op`) y V (`sem_post_op`) sobre el semáforo.

En el código 3.4 está el ciclo que crea a los hijos. `fork()` devuelve 0 en el proceso hijo, el PID del hijo en el padre y un valor negativo si falla; por eso el bloque `if (pid == 0)` es exclusivo de los hijos. Como la variable `p` del ciclo se copia en cada hijo, éste sabe qué armónico le toca. Los coeficientes a_n y b_n son las expresiones (3.3) y (3.4), y el factor $(-1)^{n+1}$ se obtiene comprobando la paridad de $n + 1$. La acumulación queda encerrada entre P y V : aunque los cien hijos terminen de calcular casi al mismo tiempo, el semáforo los deja entrar de uno en uno, de modo que ninguna suma se pierde. El `exit` final es indispensable, pues sin él cada hijo continuaría el ciclo y crearía nuevos procesos.

```

1 for (int p = 1; p <= NUM_PROCESOS; p++) {
2     pid_t pid = fork();
3     if (pid == 0) { // --- proceso hijo ---
4         int n = p; // el hijo p calcula el armónico n = p
5         double local_terms[NUM_PUNTOS];
6         double signo = ((n + 1) % 2 == 0) ? 1.0 : -1.0; // (-1)^(n+1)
7         double an = (10.0 * signo) / (3.0 * n * n);
8         double bn = (2.0 * signo) / (3.0 * n);
9
10        for (int i = 0; i < NUM_PUNTOS; i++) { // cálculo local, sin sincronizar
11            double x = x_values[i];

```

```

12     local_terms[i] = an * cos(n * x) + bn * sin(n * x);
13 }
14
15 sem_wait_op(sem_id);           // P: entrar a la sección crítica
16 for (int i = 0; i < NUM_PUNTOS; i++) {
17     f_x[i] += local_terms[i];  // suma en memoria compartida
18 }
19 sem_post_op(sem_id);          // V: salir de la sección crítica
20
21 shmdt(f_x);
22 exit(EXIT_SUCCESS);           // el hijo no debe seguir el ciclo
23 }
24 }

```

Código 3.4. Creación de los 100 hijos: cálculo local del armónico y acumulación protegida por el semáforo.

Por último, el padre (código 3.5) llama cien veces a `wait()`: cada llamada recoge el estado de terminación de un hijo, lo que evita que queden procesos zombis. Con los 64 resultados ya acumulados, el padre los escribe en `fourier_aaron_64puntos.csv` (con `fprintf`, seis decimales) y libera los recursos con `IPC_RMID`, ya que el núcleo no los elimina por sí solo.

```

1 for (int i = 0; i < NUM_PROCESOS; i++) {
2     wait(NULL);                // recoge a cada hijo: sin zombis
3 }
4 /* ... se escribe f_x[] en fourier_aaron_64puntos.csv ... */
5 shmdt(f_x);
6 shmctl(shm_id, IPC_RMID, NULL); // elimina la memoria compartida
7 semctl(sem_id, 0, IPC_RMID);   // elimina el semáforo

```

Código 3.5. El padre espera a los hijos y libera los recursos IPC.

Compilación y ejecución

Las evidencias de esta sección se obtuvieron en un equipo Apple Silicon (arquitectura `arm64`) con macOS, donde el comando `gcc` corresponde al compilador Apple clang 21 de las herramientas de línea de comandos del sistema; la ficha técnica del equipo está en la sección 3.3.1. El programa sólo usa llamadas POSIX y System V IPC, que son las mismas que ofrece el núcleo de GNU/Linux. La única diferencia entre ambas plataformas es la unión `semun` que exige `semctl`: la biblioteca glibc de GNU/Linux no la declara y el programa debe definirla, mientras que el encabezado `<sys/sem.h>` de macOS ya la trae y redefinirla produce un error de compilación. Por eso la definición va protegida como muestra el código 3.6, de modo que el mismo archivo fuente sea válido en ambos sistemas.

```

1 #if defined(__linux__)
2 union semun {
3     int val;
4     struct semid_ds *buf;
5     unsigned short *array;

```

```

6 };
7 #endif

```

Código 3.6. Definición de `union semun` sólo en GNU/Linux.

El código 3.7 muestra la compilación con `-Wall` y optimización `-O2`, enlazando la biblioteca matemática con `-lm`. La compilación termina sin errores ni advertencias: el compilador no imprime ningún mensaje y devuelve el código de salida cero. Como control adicional, el mismo archivo se compiló con `-Wall -Wextra -pedantic`, que tampoco produjo mensajes.

```

$ gcc --version | head -1
Apple clang version 21.0.0 (clang-2100.1.1.101)
$ gcc -Wall -O2 fourier_aaron.c -o fourier_aaron -lm
$ echo $?
0

```

Código 3.7. Compilación del programa sin errores ni advertencias.

La ejecución (código 3.8) muestra primero al padre y su PID, después una línea por cada hijo con su PID y el armónico que procesó, y al final la confirmación de que los cien hijos terminaron y de que se liberaron la memoria compartida y el semáforo. En esta ejecución los PIDs de los hijos resultaron consecutivos, porque el padre los crea uno tras otro. El orden en que los hijos imprimen su línea no está garantizado, pues depende del planificador; en esta ejecución coincidió con el orden de creación.

```

$ ./fourier_aaron
=====
CÓMPUTO PARALELO - CAPÍTULO 3: PROGRAMACIÓN CON PROCESOS
Integrante: Aarón Ugalde Téllez
Evaluación de Serie de Fourier con 100 Procesos Hijos (64 Puntos)
=====

[PADRE | PID: 6729] Creando 100 procesos hijos...

[HIJO #001 | PID: 6730] Armónico n = 1 procesado para los 64 puntos.
[HIJO #002 | PID: 6731] Armónico n = 2 procesado para los 64 puntos.
[HIJO #003 | PID: 6732] Armónico n = 3 procesado para los 64 puntos.
...
[HIJO #098 | PID: 6827] Armónico n = 98 procesado para los 64 puntos.
[HIJO #099 | PID: 6828] Armónico n = 99 procesado para los 64 puntos.
[HIJO #100 | PID: 6829] Armónico n = 100 procesado para los 64 puntos.

[PADRE] Los 100 procesos hijos han completado el cálculo.
[PADRE] Resultados guardados en el archivo 'fourier_aaron_64puntos.csv'.
[PADRE] Recursos IPC (SHM y semáforos) liberados correctamente.
=====

```

Código 3.8. Ejecución del programa con 100 procesos hijos.

La figura 3.1 es la captura de esa sesión en la terminal. Al comparar la salida de ejecuciones sucesivas, el archivo CSV resultó idéntico byte a byte: aunque el orden en que los hijos entran a la sección crítica cambia de una ejecución a otra, la diferencia que eso introduce al sumar números de punto flotante es de un orden muy inferior a los seis decimales con que se escribe el archivo.

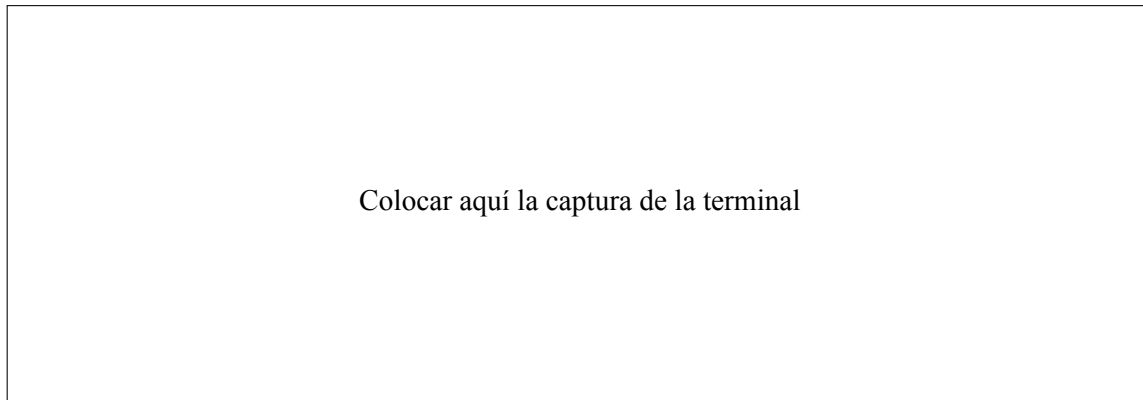


Figura 3.1. Compilación y ejecución del programa con 100 procesos hijos en la terminal.

Resultados obtenidos y gráfica

El padre escribe los 64 valores acumulados en `fourier_aaron_64puntos.csv`, un archivo de texto plano con dos columnas, `x` y `f_fourier`, una fila por nodo y seis decimales. Ese archivo se importó en la hoja de cálculo de la fase anterior para generar la gráfica de la figura 3.2.

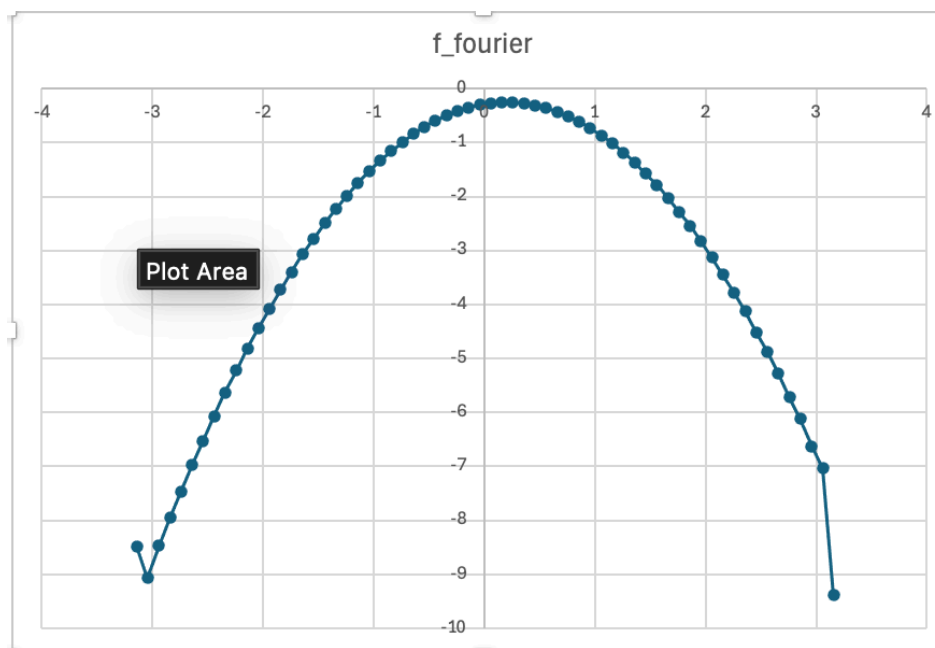


Figura 3.2. Gráfica de $F_{100}(x)$ calculada en C con 100 procesos hijos, en los 64 nodos de $x = -3.1416$ a $x = 3.1584$.

La gráfica cuenta cuatro cosas:

- **Forma parabólica.** La curva es una parábola invertida. Su vértice no está en el origen sino en $x = \frac{1}{5}$, desplazado por el término lineal $\frac{1}{3}x$, y allí $f\left(\frac{1}{5}\right) = -0.2524$. Entre los nodos, el máximo de la serie es -0.2530 en $x = 0.1584$. Toda la curva es negativa, porque el máximo de la parábola ya es menor que cero.
- **Muestreo discreto.** Cada marcador es uno de los 64 nodos; la línea sólo los une. En el interior del intervalo los nodos caen sobre la parábola y la curva es suave.
- **Extremo izquierdo: fenómeno de Gibbs.** La función original vale $f(-\pi) \approx -9.5576$ y $f(\pi) \approx -7.4632$, de modo que al repetirla periódicamente aparece un salto de $-\frac{2\pi}{3} \approx -2.0944$ en cada múltiplo impar de π [4]. Una suma de senos y cosenos no puede reproducir un salto: en el salto converge al punto medio, $-\frac{5\pi^2}{6} - \frac{2}{7} \approx -8.5104$, y cerca de él oscila. Eso es lo que se ve en los primeros nodos: el nodo $x = -3.1416$ vale -8.4767 , muy cerca del punto medio y lejos de $f(-\pi)$; el siguiente baja a -9.0642 , por debajo de la función (-9.0090), y el tercero vuelve a quedar ligeramente por encima (-8.4666 frente a -8.4771). Las oscilaciones se amortiguan rápidamente al alejarse del salto.
- **Extremo derecho.** El último nodo, $x = 3.1584$, queda 0.0168 más allá de π , es decir, en el periodo siguiente, donde la serie ya describe el tramo que comienza en $-\pi$. Por eso la curva cae de -7.0347 a -9.3902 en el último paso: no es un error del programa sino el salto de la extensión periódica, cuyo valor del lado izquierdo, $f(3.1584 - 2\pi) = f(-3.1248) \approx -9.4642$, la serie de 100 armónicos ya casi alcanza.

Comparación con las fases anteriores

La tabla 3.1 reúne, para diez de los 64 nodos, la función original $f(x)$ de los capítulos 1 y 2, la aproximación $F_{10}(x)$ de la hoja de cálculo del capítulo 2 y la aproximación $F_{100}(x)$ del programa en C. Los coeficientes son los mismos en las tres fases, pues provienen del desarrollo analítico del capítulo 1; lo único que cambia es el número de armónicos y la forma de calcularlos.

Tabla 3.1. Función original y aproximaciones con 10 armónicos (Excel) y 100 armónicos (C, 100 procesos), en diez de los 64 nodos.

x	$f(x)$	$F_{10}(x)$	$F_{100}(x)$	$ f - F_{10} $	$ f - F_{100} $
-3.1416	-9.5576	-8.1931	-8.4767	1.3645	1.0809
-3.0416	-9.0090	-8.6570	-9.0642	0.3520	0.0552
-2.1416	-4.8216	-4.8333	-4.8147	0.0117	0.0069
-1.0416	-1.5370	-1.5734	-1.5402	0.0364	0.0031
-0.0416	-0.3010	-0.3012	-0.3038	0.0002	0.0028
0.9584	-0.7317	-0.6967	-0.7349	0.0350	0.0032
1.9584	-2.8290	-2.8769	-2.8342	0.0479	0.0051
2.9584	-6.5930	-6.7042	-6.6239	0.1112	0.0309
3.0584	-7.0611	-7.5498	-7.0347	0.4887	0.0264
3.1584	-7.5458	-8.3003	-9.3902	0.7545	1.8444

En el interior del intervalo, F_{100} se acerca a f mucho más que F_{10} : los errores pasan de centésimas a milésimas. Para medirlo se usó el error cuadrático medio (MSE, *mean squared error*) entre cada aproximación y $f(x)$, calculado sobre distintos subconjuntos de nodos (tabla 3.2).

Tabla 3.2. Error cuadrático medio respecto de $f(x)$ según la región de nodos considerada.

Región	MSE de F_{10}	MSE de F_{100}	Mejora
$ x \leq 2.5$ (50 nodos)	1.40×10^{-3}	1.33×10^{-5}	≈ 106 veces
$ x \leq \pi$ (62 nodos)	9.63×10^{-3}	1.08×10^{-4}	≈ 89 veces
Los 64 nodos	4.73×10^{-2}	7.15×10^{-2}	empeora

La última fila se explica por el último nodo, que queda fuera de $[-\pi, \pi]$: ahí $f(x)$ sigue el polinomio (-7.5458), mientras que F_{100} ya describe el periodo siguiente (-9.3902), de modo que ese solo nodo aporta un error de 1.84 y domina el promedio. Comparar la serie con f en ese punto no es significativo, y por eso la comparación razonable es la de las filas anteriores. Cerca del salto el error también disminuye con más armónicos, pero mucho más despacio: en $x = -3.1416$ pasa de 1.36 a 1.08, porque la serie converge allí al punto medio del salto y no a f . Este es el mismo comportamiento que se observó con diez armónicos en el capítulo 2, sólo que ahora la región afectada es diez veces más estrecha.

Por último, se comprobó que la acumulación paralela es correcta. Los 64 valores del programa en C se contrastaron con una evaluación serial e independiente de la misma suma de 100 armónicos (script del anexo 3.5.2): el error cuadrático medio entre ambos es de 7.4×10^{-14} y la mayor diferencia absoluta es de 5.0×10^{-7} , es decir, media unidad del último decimal del archivo CSV. Esto confirma que el semáforo evitó las condiciones de carrera: si se hubiera perdido alguna de las cien contribuciones, la mayor diferencia sería como mínimo del orden de 10^{-3} , pues incluso el armónico más pequeño, $n = 100$, aporta términos de hasta $b_{100} \approx 6.7 \times 10^{-3}$.

3.3 Análisis de la implementación con procesos

3.3.1 Implementación de Aarón Ugalde Téllez

La tabla 3.3 resume las características de la implementación de esta fase.

Tabla 3.3. Características de la implementación con procesos.

Característica	Especificación
Lenguaje	C, compilado con Apple clang 21.0.0 (invocado como <code>gcc</code>)
Sistema operativo	macOS 26.5 (Darwin, arm64)
Procesos	1 padre y 100 hijos creados con <code>fork()</code>
Memoria compartida	System V: un segmento de 512 bytes (64 valores <code>double</code>) con <code>shmget</code> , <code>shmat</code> , <code>shmdt</code> y <code>shmctl</code>
Sincronización	Un semáforo System V binario (valor inicial 1) con <code>semget</code> , <code>semctl</code> y <code>semop</code>
Reparto del trabajo	Un armónico por proceso ($n = p$), evaluado en los 64 nodos: 128 evaluaciones trigonométricas por hijo
Sección crítica	64 sumas en punto flotante por proceso; 100 entradas en total
Compilación	<code>gcc -Wall -O2 fourier_aaron.c -o fourier_aaron -lm</code> , sin errores ni advertencias
Salida	Archivo CSV de 64 filas (<code>x</code> , <code>f_fourier</code>) con seis decimales
Tiempo de ejecución	Alrededor de 10 ms de tiempo de reloj (cinco ejecuciones medidas con <code>time</code> , resolución de 10 ms)

Lenguaje y técnica utilizados

El lenguaje C es el adecuado para esta fase porque da acceso directo a las llamadas al sistema que se estudian (`fork`, `shmget`, `semop`) sin capas intermedias y se compila a código nativo, de modo que el costo que se observa es el del sistema operativo y no el de un intérprete [3]. La biblioteca matemática aporta `sin` y `cos`, y se enlaza con `-lm`.

La técnica es el paralelismo a nivel de procesos con memoria compartida explícita. Cada proceso hijo tiene su propio espacio de direcciones, de modo que nada se comparte si el programa no lo pide: por eso fue necesario crear el segmento de memoria compartida y un semáforo para proteger la escritura sobre él. Es una técnica más pesada que los hilos, porque crear un proceso implica duplicar las estructuras que describen su memoria, pero ofrece un aislamiento más fuerte: un error en un hijo no puede corromper la memoria de los demás [2].

Cantidad de procesos

El programa crea 100 procesos hijos, que con el padre suman 101, sobre un equipo con 10 núcleos: a cada núcleo le tocan alrededor de diez procesos, de los cuales sólo uno se ejecuta a la vez y el planificador

intercala a los demás. El sistema lo permite sin dificultad, pues el límite de procesos por usuario del equipo es 2666. Sin embargo, el trabajo de cada hijo es mínimo: 128 evaluaciones de seno y coseno, que toman microsegundos. Por ello, el tiempo total de alrededor de 10 ms corresponde sobre todo a crear, planificar y esperar a los procesos, no a calcular, y es de esperar que una versión serial del mismo cálculo, sin procesos, resulte más rápida. Esto no le resta valor a la fase, cuyo propósito es aprender a coordinar procesos con memoria compartida y semáforos; la ganancia real de velocidad llegaría con un trabajo por proceso mucho mayor que el costo de crearlo.

Equipo de cómputo

La tabla 3.4 presenta las características del equipo en el que se compiló y ejecutó el programa. Los valores de núcleos, memoria y cachés los reporta el propio sistema (`sysctl` y `system_profiler`); Apple no publica la frecuencia de reloj ni la velocidad de sus cachés, por lo que para la frecuencia se cita una fuente externa.

Tabla 3.4. Especificaciones del equipo de cómputo empleado.

Componente	Detalle
Equipo	MacBook Air (identificador <code>Mac17,3</code>)
Procesador	Apple M5, arquitectura <code>arm64</code>
Núcleos	10: 4 de alto rendimiento (“super”) y 6 de eficiencia; un hilo por núcleo, sin multihilo simultáneo
Frecuencia	No publicada por Apple; hasta 4.4 GHz según EveryMac [5]
Memoria RAM	16 GB de memoria unificada LPDDR5 (Micron)
Ancho de banda de memoria	153 GB/s, según Apple [6]
Caché L1, núcleos de alto rendimiento	192 KB de instrucciones y 128 KB de datos, por núcleo
Caché L1, núcleos de eficiencia	128 KB de instrucciones y 64 KB de datos, por núcleo
Caché L2	16 MB compartidos por los 4 núcleos de alto rendimiento; 6 MB compartidos por los 6 de eficiencia
Caché L3	No reportada por el sistema
Línea de caché / página de memoria	128 bytes / 16 KB

3.4 Conclusiones del capítulo

3.4.1 Conclusión — Aarón Ugalde Téllez

Esta práctica fue un ejercicio completo de cómputo paralelo: me permitió tender un puente entre el rigor de las series de Fourier y los retos prácticos de programar procesos concurrentes a bajo nivel en un sistema POSIX.

Desde el análisis numérico, trabajar con $f(x) = -\frac{5}{6}x^2 + \frac{1}{3}x - \frac{2}{7}$ a lo largo de las tres fases me dio una

comprensión más profunda de cómo se comporta la serie. Pasé de los cálculos analíticos a mano del capítulo 1 a la verificación en hoja de cálculo del capítulo 2 y, finalmente, a la reconstrucción con 100 procesos hijos en este capítulo. Ver cómo la superposición de 100 armónicos reproduce la curvatura cuadrática dentro de $[-\pi, \pi]$, y cómo en los extremos aparece el fenómeno de Gibbs, me dejó claro que la serie no converge a la función sino a su extensión periódica. El último nodo, que por quedar apenas fuera del intervalo ya pertenece al periodo siguiente, lo mostró de forma muy concreta. Esta base es la que sostiene gran parte del procesamiento de señales y de varias técnicas de aprendizaje automático.

En el diseño de software concurrente, las llamadas al sistema convirtieron conceptos teóricos de sistemas operativos en herramientas de ingeniería. Crear 100 procesos con `fork()` hizo evidente la sobrecarga de duplicar el contexto de ejecución de un proceso, frente a los hilos: el cálculo de cada hijo dura microsegundos y el tiempo total lo domina la creación de procesos. La memoria compartida System V (`shmget`, `shmat`) me mostró cómo se rompe de forma controlada el aislamiento que el núcleo impone entre procesos.

El reto más valioso fue la sincronización con semáforos. Acumular 100 contribuciones en un mismo vector sin exclusión mutua habría producido condiciones de carrera; con las operaciones atómicas P y V (`semop`) la acumulación resultó correcta, y lo comprobé al comparar la salida paralela con una evaluación serial, que coincidió hasta el sexto decimal. Coordinar a los 100 procesos sin bloqueos mutuos ni procesos zombis, y liberar al final los recursos IPC, exigió disciplina en el control de flujo. Incluso un detalle como la unión `semun`, que glibc deja al programador y macOS trae en su encabezado, me recordó que la portabilidad entre sistemas POSIX requiere atención.

Como estudiante de Ingeniería en Inteligencia Artificial, estoy acostumbrado a abstraer los algoritmos complejos en bibliotecas de alto nivel; no obstante, esta fase me recordó la importancia de entender las capas inferiores: la planificación del núcleo, la memoria y el costo real de la concurrencia. Dominar la programación paralela a bajo nivel en C es una base indispensable para optimizar flujos de datos, diseñar motores de inferencia eficientes y construir arquitecturas distribuidas que escalen.

3.5 Anexos

En esta sección se colocan los códigos generados en la fase 3 (programación con procesos, capítulo 3).

3.5.1 Fase 3: código de `fourier_aaron.c`

Programa de Aarón Ugalde Téllez que calcula la Serie de Fourier con 100 procesos hijos, memoria compartida y semáforos System V.

```

1  /*
2  *
3  *  =====
4  *  INSTITUTO POLITÉCNICO NACIONAL - ESCUELA SUPERIOR DE CÓMPUTO
5  *  Cómputo Paralelo - Grupo 6BV1
6  *  Capítulo 3: Programación con Procesos
7  *
8  *  Autor: Aarón Ugalde Téllez

```

```

8  * Función asignada:  $f(x) = -5/6 * x^2 + (1/3) * x - 2/7$ ,  $x$  en  $[-\pi, \pi]$ 
9  * Configuración: 100 Procesos Hijos, Memoria Compartida, Semáforos System V.
10 * Puntos de evaluación: 64 nodos  $x = -3.1416 + 0.1*k$ ,  $k = 0..63$  (los mismos
11 *      nodos de la hoja de cálculo del Capítulo 2).
12 *
13 * Compilación: gcc -Wall -O2 fourier_aaron.c -o fourier_aaron -lm
14 *
15  */
16
17 #include <stdio.h>
18 #include <stdlib.h>
19 #include <unistd.h>
20 #include <math.h>
21 #include <sys/ipc.h>
22 #include <sys/shm.h>
23 #include <sys/sem.h>
24 #include <sys/wait.h>
25
26 #define NUM_PROCESOS 100 // Exactly 100 child processes
27 #define NUM_PUNTOS 64 // Array size corresponding to specified x values
28
29 // Arreglo estático con los 64 valores exactos de x
30 static const double x_values[NUM_PUNTOS] = {
31     -3.1416, -3.0416, -2.9416, -2.8416, -2.7416, -2.6416, -2.5416, -2.4416,
32     -2.3416, -2.2416, -2.1416, -2.0416, -1.9416, -1.8416, -1.7416, -1.6416,
33     -1.5416, -1.4416, -1.3416, -1.2416, -1.1416, -1.0416, -0.9416, -0.8416,
34     -0.7416, -0.6416, -0.5416, -0.4416, -0.3416, -0.2416, -0.1416, -0.0416,
35     0.0584, 0.1584, 0.2584, 0.3584, 0.4584, 0.5584, 0.6584, 0.7584,
36     0.8584, 0.9584, 1.0584, 1.1584, 1.2584, 1.3584, 1.4584, 1.5584,
37     1.6584, 1.7584, 1.8584, 1.9584, 2.0584, 2.1584, 2.2584, 2.3584,
38     2.4584, 2.5584, 2.6584, 2.7584, 2.8584, 2.9584, 3.0584, 3.1584
39 };
40
41 // Estructura requerida para semctl. glibc (GNU/Linux) no la declara y debe
42 // definirla el programa; macOS ya la trae en <sys/sem.h>, por eso la guarda.
43 #if defined(__linux__)
44 union semun {
45     int val;
46     struct semid_ds *buf;
47     unsigned short *array;
48 };
49 #endif
50
51 // Operación P (Wait / Bloquear semáforo)
52 void sem_wait_op(int sem_id) {
53     struct sembuf sb = {0, -1, 0};

```

```

54     semop(sem_id, &sb, 1);
55 }
56
57 // Operación V (Signal / Liberar semáforo)
58 void sem_post_op(int sem_id) {
59     struct sembuf sb = {0, 1, 0};
60     semop(sem_id, &sb, 1);
61 }
62
63 int main(void) {
64     printf("=====\n");
65     printf(" CÓMPUTO PARALELO - CAPÍTULO 3: PROGRAMACIÓN CON PROCESOS\n");
66     printf(" Integrante: Aarón Ugalde Téllez\n");
67     printf(" Evaluación de Serie de Fourier con 100 Procesos Hijos (%d\n", NUM_PUNTOS);
68     printf("=====\n\n");
69
70     // 1. Crear Memoria Compartida para guardar el arreglo de los 64 resultados
71     size_t shm_size = NUM_PUNTOS * sizeof(double);
72     int shm_id = shmget(IPC_PRIVATE, shm_size, IPC_CREAT | 0666);
73     if (shm_id < 0) {
74         perror("Error al crear la memoria compartida (shmget)");
75         exit(EXIT_FAILURE);
76     }
77
78     // Acoplar la memoria compartida al espacio de direcciones
79     double *f_x = (double *)shmat(shm_id, NULL, 0);
80     if (f_x == (double *)-1) {
81         perror("Error al acoplar la memoria compartida (shmat)");
82         exit(EXIT_FAILURE);
83     }
84
85     // 2. Inicializar la memoria compartida con el término constante (a0/2)
86     // a0/2 = -5/18 * pi^2 - 2/7
87     double a0 = (-5.0 / 18.0) * (M_PI * M_PI) - (2.0 / 7.0);
88     for (int i = 0; i < NUM_PUNTOS; i++) {
89         f_x[i] = a0;
90     }
91
92     // 3. Crear e inicializar el semáforo System V
93     int sem_id = semget(IPC_PRIVATE, 1, IPC_CREAT | 0666);
94     if (sem_id < 0) {
95         perror("Error al crear el semáforo (semget)");
96         exit(EXIT_FAILURE);
97     }
98
99     union semun arg;

```

```

100  arg.val = 1; // Inicializado en 1 (sección crítica libre)
101  if (semctl(sem_id, 0, SETVAL, arg) < 0) {
102      perror("Error al inicializar el semáforo (semctl)");
103      exit(EXIT_FAILURE);
104  }
105
106  printf("[PADRE | PID: %d] Creando %d procesos hijos...\n\n", getpid(),
        NUM_PROCESOS);
107  fflush(stdout); // vacía el búfer: si no, cada hijo heredaría (y
        repetiría) este texto
108
109  // 4. Crear los 100 procesos hijos
110  for (int p = 1; p <= NUM_PROCESOS; p++) {
111      pid_t pid = fork();
112
113      if (pid < 0) {
114          perror("Error al crear proceso hijo (fork)");
115          exit(EXIT_FAILURE);
116      }
117
118      if (pid == 0) {
119          // === PROCESO HIJO ===
120          int n = p; // El hijo 'p' procesa el armónico n = p
121          double local_terms[NUM_PUNTOS];
122
123          // Coeficientes an y bn de la función de Aarón Ugalde
124          double signo = ((n + 1) % 2 == 0) ? 1.0 : -1.0; // (-1)^(n+1)
125          double an = (10.0 * signo) / (3.0 * n * n);
126          double bn = (2.0 * signo) / (3.0 * n);
127
128          // Calcular término n para cada uno de los 64 puntos x especificados
129          for (int i = 0; i < NUM_PUNTOS; i++) {
130              double x = x_values[i];
131              local_terms[i] = an * cos(n * x) + bn * sin(n * x);
132          }
133
134          // --- SECCIÓN CRÍTICA ---
135          sem_wait_op(sem_id); // Entrar a sección crítica
136
137          for (int i = 0; i < NUM_PUNTOS; i++) {
138              f_x[i] += local_terms[i];
139          }
140
141          sem_post_op(sem_id); // Salir de sección crítica
142          // -----
143
144          printf("[HIJO #03d | PID: %d] Armónico n = %3d procesado para los

```

```

145         %d puntos.\n", n, getpid(), n, NUM_PUNTOS);
146     shmdt(f_x);
147     exit(EXIT_SUCCESS);
148 }
149 }
150
151 // 5. El proceso padre espera a que finalicen los 100 hijos
152 for (int i = 0; i < NUM_PROCESOS; i++) {
153     wait(NULL);
154 }
155
156 printf("\n[PADRE] Los %d procesos hijos han completado el cálculo.\n",
157        NUM_PROCESOS);
158
159 // 6. Generar archivo CSV con los resultados exactos
160 const char *filename = "fourier_aaron_64puntos.csv";
161 FILE *fp = fopen(filename, "w");
162 if (fp == NULL) {
163     perror("Error al crear el archivo CSV de salida");
164     exit(EXIT_FAILURE);
165 }
166 fprintf(fp, "x,f_fourier\n");
167 for (int i = 0; i < NUM_PUNTOS; i++) {
168     fprintf(fp, "%.4f,%.6f\n", x_values[i], f_x[i]);
169 }
170 fclose(fp);
171
172 printf("[PADRE] Resultados guardados en el archivo '%s'.\n", filename);
173
174 // 7. Liberar memoria compartida y semáforos
175 shmdt(f_x);
176 shmctl(shm_id, IPC_RMID, NULL);
177 semctl(sem_id, 0, IPC_RMID);
178
179 printf("[PADRE] Recursos IPC (SHM y semáforos) liberados
180        correctamente.\n");
181 printf("=====\n");
182 return 0;
183 }

```

Código 3.9. Programa fourier_aaron.c.

3.5.2 Fase 3: código de `verificacion_serie.py`

Script en Python de Aarón Ugalde Téllez que evalúa la misma suma de forma serial y calcula los errores de las tablas 3.1 y 3.2.

```

1  #!/usr/bin/env python3
2  """Verificación serial de la Serie de Fourier de  $f(x) = -5/6 x^2 + x/3 - 2/7$ .
3
4  Lee el CSV que produce fourier_aaron.c y lo compara con:
5  *  $f(x)$  : la función original (Capítulo 1),
6  *  $F_{10}(x)$  : la suma de 10 armónicos de la hoja de cálculo (Capítulo 2),
7  *  $F_{100}(x)$ : la suma de 100 armónicos evaluada en forma serial (sin procesos).
8  Uso: python3 verificacion_serie.py fourier_aaron_64puntos.csv
9  """
10 import csv
11 import math
12 import sys
13
14 PI = math.pi
15
16
17 def f(x):
18     return -5 / 6 * x * x + x / 3 - 2 / 7
19
20
21 def serie(x, armonicos):
22     """ $a_0/2$  + suma de los primeros 'armonicos' términos (mismas fórmulas que
23     el C)."""
24     s = -5 * PI ** 2 / 18 - 2 / 7
25     for n in range(1, armonicos + 1):
26         signo = 1.0 if (n + 1) % 2 == 0 else -1.0
27         s += 10 * signo / (3 * n * n) * math.cos(n * x)
28         s += 2 * signo / (3 * n) * math.sin(n * x)
29     return s
30
31 def mse(a, b, idx):
32     return sum((a[i] - b[i]) ** 2 for i in idx) / len(idx)
33
34
35 ruta = sys.argv[1] if len(sys.argv) > 1 else "fourier_aaron_64puntos.csv"
36 filas = list(csv.DictReader(open(ruta)))
37 xs = [float(r["x"]) for r in filas]
38 f_c = [float(r["f_fourier"]) for r in filas] # resultado del programa en C
39 f_ref = [f(x) for x in xs]
40 f_10 = [serie(x, 10) for x in xs]
41 f_100 = [serie(x, 100) for x in xs]

```

```

42
43 todos = list(range(len(xs)))
44 interior = [i for i in todos if abs(xs[i]) <= 2.5] # lejos de la
      discontinuidad
45 dentro = [i for i in todos if abs(xs[i]) <= PI] # nodos dentro de [-pi, pi]
46
47 print("C vs serie serial (100 armónicos): MSE = %.3e, |dif| máx = %.3e"
48       % (mse(f_c, f_100, todos), max(abs(a - b) for a, b in zip(f_c, f_100))))
49 for nombre, serie_k in (("F10 (Excel)", f_10), ("F100 (C) ", f_c)):
50     print("%s vs f: MSE |x|<=2.5 = %.3e | MSE |x|<=pi = %.3e | MSE 64 nodos =
      %.3e"
51           % (nombre, mse(serie_k, f_ref, interior), mse(serie_k, f_ref, dentro),
52               mse(serie_k, f_ref, todos)))
53
54 print("\n x      f(x)      F10      F100(C)")
55 for k in (0, 1, 2, 10, 21, 31, 32, 41, 51, 61, 62, 63):
56     print("%8.4f %10.6f %10.6f %10.6f" % (xs[k], f_ref[k], f_10[k], f_c[k]))

```

Código 3.10. Script verificacion_serie.py.

Referencias

- [1] A. Silberschatz, P. B. Galvin y G. Gagne, *Operating System Concepts*, 10.^a ed. Hoboken, NJ: Wiley, 2018.
- [2] A. S. Tanenbaum y H. Bos, *Modern Operating Systems*, 4.^a ed. Boston, MA: Pearson, 2014.
- [3] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA: No Starch Press, 2010.
- [4] E. Kreyszig, *Advanced Engineering Mathematics*, 10.^a ed. Jefferson City, MO, USA: John Wiley & Sons, 2011.
- [5] «Apple MacBook Air “M5” 10 CPU/10 GPU 13” Specs,» EveryMac.com, visitado 30 de sep. de 2026. dirección: <https://everymac.com/systems/apple/macbook-air/specs/macbook-air-m5-10-core-cpu-10-core-gpu-13-2026-specs.html>
- [6] Apple Inc. «MacBook Air: Technical Specifications,» Apple, visitado 30 de sep. de 2026. dirección: <https://www.apple.com/macbook-air/specs/>